Two Colorable

AlgoExpert · Medium · Graphs

The problem

- Undirected connected graph as adjacency list; can it be 2-coloured (bipartite)?

Intuition

- A graph is two-colourable if BFS or DFS can colour neighbours alternately without conflict.

Brute force vs optimal

- Brute force: try all 2^n colourings
- BFS/DFS assigning alternating colours → O(V + E)
- Bipartite ⇔ no odd cycle

Approach

- 1. Colour the start
- 2. Give each neighbour the opposite colour
- 3. A neighbour with the same colour means false

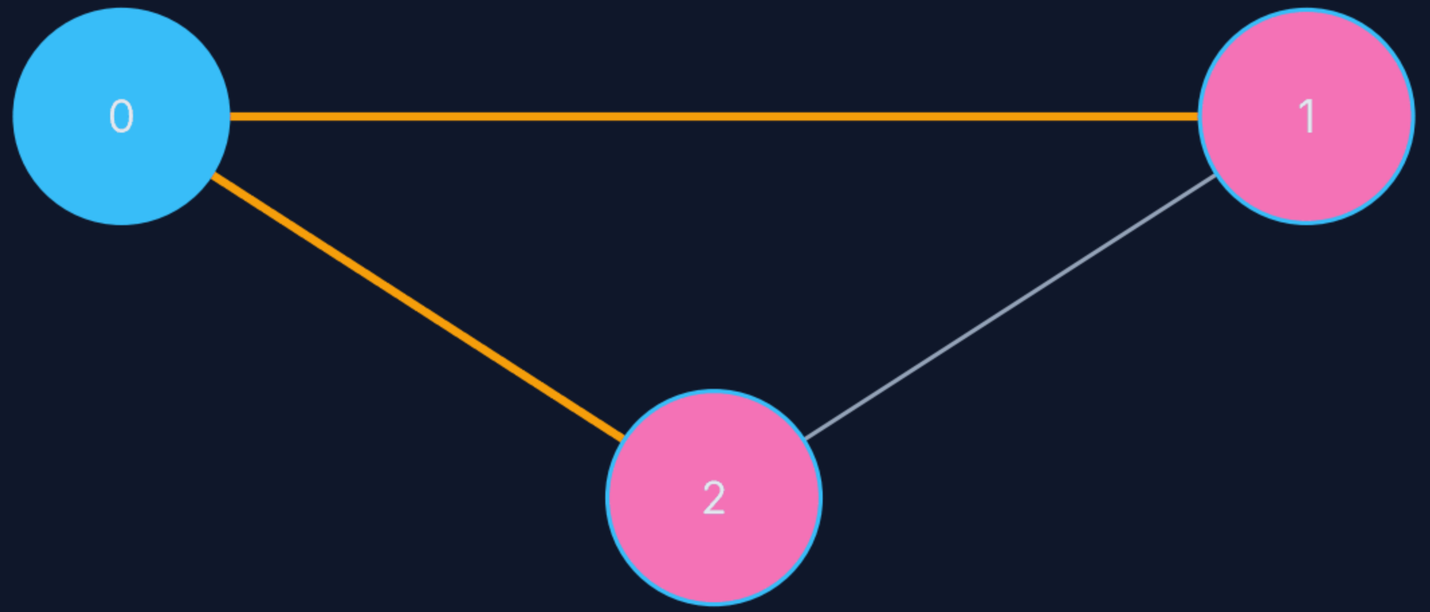
Worked example




```
0: blue
```

Colour 0 blue

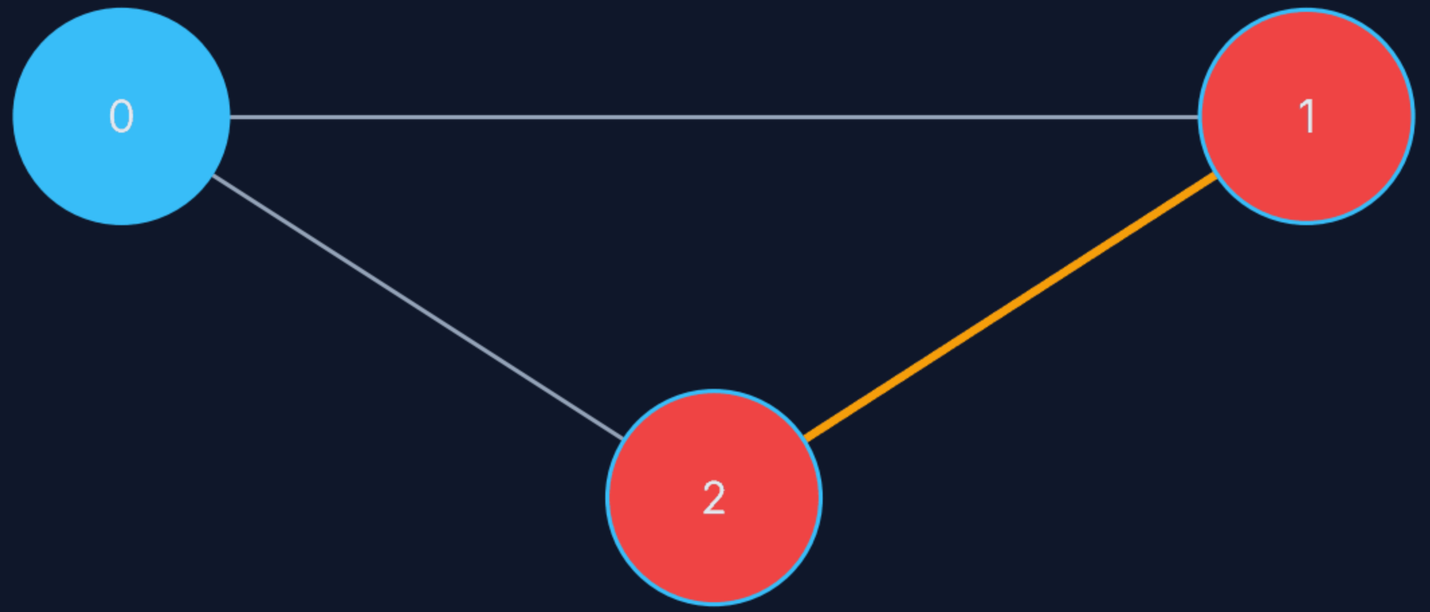
Worked example




```
1, 2: pink (opposite of 0)
```

Neighbours 1 and 2 get pink

Worked example




```
edge 1–2: both pink ✗
```

Edge 1–2 joins two pink nodes → not 2-colourable

Java solution

```java
public class TwoColorable {
    public static boolean twoColorable(int[][] edges) {
        int[] color = new int[edges.length];
        java.util.ArrayDeque<Integer> st = new java.util.ArrayDeque<>();
        for (int s = 0; s < edges.length; s++) {
            if (color[s] != 0) continue;
            color[s] = 1;
            st.push(s);
            while (!st.isEmpty()) {
                int v = st.pop();
                for (int w : edges[v]) {
                    if (color[w] == 0) { color[w] = -color[v]; st.push(w); }
                    else if (color[w] == color[v]) return false;
                }
            }
        }
        return true;
    }
}
```

Complexity

- Time: O(v + e)
- Space: O(v)

Edge cases & pitfalls

- Disconnected graphs: start from every uncoloured node
- Self-loop → false
- Undirected edges appear in both lists

Interview tips & follow-ups

- Explain the odd-cycle characterisation
- Applications: scheduling into two groups

Recap

- A graph is two-colourable if BFS or DFS can colour neighbours alternately without conflict.
- Complexity: O(v + e)